Phân đoạn đối tượng bằng hình thái học toán học và Watershed

BÁO CÁO BÀI TẬP LỚN MÔN XỬ LÝ ẢNH

Học viện Công nghệ Bưu chính Viễn thông

Nhóm 1

Kiều Trường Giang — B23DCCN254
Trần Đỗ Lan Phương — B23DCCN667
Giang Thùy Tiên — B23DCCN814
Phạm Văn Vỹ — B23DCCN954

Nền tảng kiểm chứng: Fiji/ImageJ và MorphoLibJ 1.6.5
Ngôn ngữ cài đặt: Python 3.11
Repository: https://github.com/jellalaz/XU_LY_ANH
Ngày tổng hợp số liệu: 08 tháng 11 năm 2026

Báo cáo nghiên cứu cách tách từng đối tượng trong ảnh hai chiều bằng hình thái học và Watershed có marker. Kết quả gồm cài đặt tự viết, kiểm chứng với thư viện và Fiji, thí nghiệm trên ảnh tổng hợp và BBBC039. Ứng dụng giao diện là công cụ hỗ trợ thực nghiệm của dự án.

Tóm tắt và mục lục

Nhóm xây dựng quy trình chuẩn hóa ảnh, tạo mặt nạ, xử lý hình thái, tính khoảng cách tới nền, tạo marker và lan nhãn bằng Watershed. Cài đặt tự viết tập trung ở các phép hình thái nhị phân, tái tạo, gán nhãn liên thông, khoảng cách Euclid và hàng đợi ưu tiên. Gaussian, Otsu, đọc ghi và thống kê sử dụng thư viện được công bố rõ.

Trên 50 ảnh test BBBC039, F1 trung bình theo ảnh của Python tự cài đạt 0,7087; Python thư viện đạt 0,7054; Fiji nhận chung đầu vào đạt 0,7188; quy trình Fiji độc lập đạt 0,8917. Các quy trình có tiền xử lý khác nhau, vì vậy số liệu này không chứng minh riêng một hàm Watershed luôn tốt hơn hàm khác. IoU tiền cảnh cao cũng chưa bảo đảm tách đúng từng vật thể.

Mục tiêu học thuật là giải thích được thuật toán, quy ước cài đặt, thiết kế kiểm chứng và nguyên nhân sai khác. Nhóm không huấn luyện mô hình học sâu. Kết quả là nghiên cứu phân đoạn hình thái, có giới hạn khi đối tượng không tách biệt về độ sáng hoặc hình dạng.

03 Bài toán và câu hỏi nghiên cứu
04 Biểu diễn ảnh và các quy ước toán học
05 Erosion và Dilation
06 Opening và Closing
07 Ví dụ tính tay và kết quả kiểm tra
08 Tái tạo hình thái
09 Cài đặt reconstruction bằng hàng đợi
10 Gán nhãn các vùng liên thông
11 Biến đổi khoảng cách Euclid
12 Tính khoảng cách bằng bao dưới của parabol
13 Marker và lọc cực đại theo h
14 Watershed có điều khiển marker
15 Mã cài đặt Watershed
16 Quy trình Python hoàn chỉnh
17 Fiji và plugin MorphoLibJ
18 Thực hành kiểm chứng Watershed trong Fiji
19 Quy trình Fiji độc lập và các khác biệt
20 Dataset và nhãn đúng
21 Chỉ số định lượng và cách ghép đối tượng
22 Thiết kế thí nghiệm và kiểm soát sai lệch
23 Kết quả đối chiếu trên 50 ảnh test
24 Phân tích ảnh hưởng của từng bước
25 Ảnh hưởng của nhiễu và làm mờ
26 Benchmark và thực nghiệm tối ưu
27 Kiểm thử tính đúng và khả năng tái lập
28 Hướng dẫn chạy phần nghiên cứu bằng lệnh
29 Kịch bản demo và kiểm tra trực tiếp
30 Giới hạn và kết luận
31 Tài liệu tham khảo
32 Đối chiếu yêu cầu và bàn giao

Các số trên đánh dấu thứ tự phần nội dung; xem số trang ở chân trang.

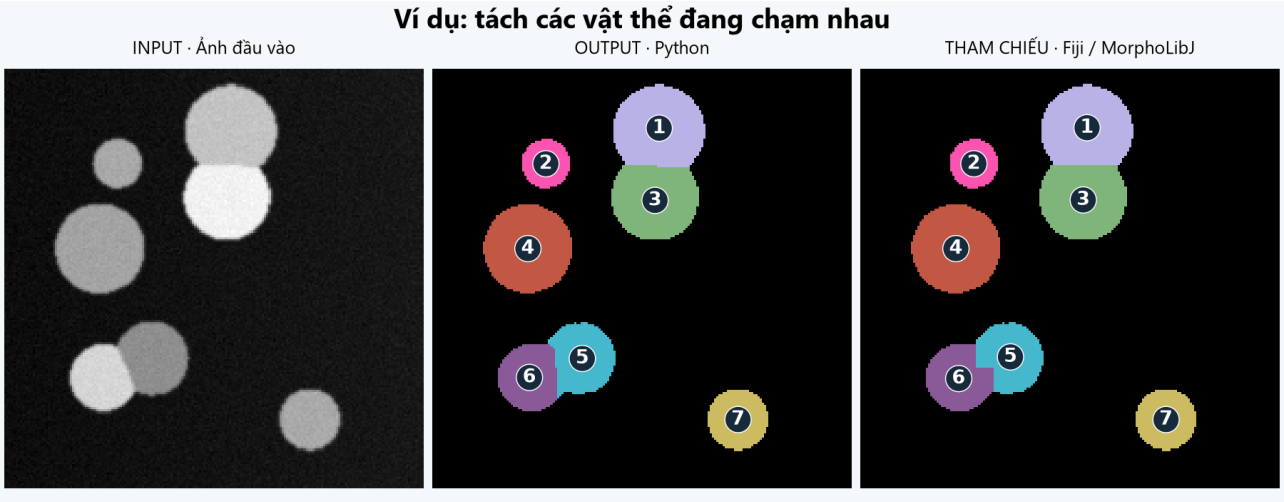
Bài toán và câu hỏi nghiên cứu

Đầu vào là ảnh xám hai chiều I có H hàng và W cột. Nếu đầu vào RGB, chương trình chuyển thành ảnh xám trước khi xử lý. Đầu ra L là ảnh nhãn nguyên không âm cùng kích thước: 0 là nền, mỗi số dương nhận diện một đối tượng. Ảnh tô màu chỉ dùng quan sát; ảnh nhãn TIFF mới là dữ liệu để tính chỉ số.

Phân đoạn tiền cảnh trả lời pixel nào thuộc đối tượng. Phân đoạn từng đối tượng còn cần phân chia các pixel tiền cảnh thành những vật thể khác nhau. Hai nhân tế bào đang chạm nhau có thể tạo một vùng liên thông nhưng phải mang hai nhãn trong kết quả mong muốn. Đếm số nhãn là một phép đo sau phân đoạn, không phải toàn bộ bài toán.

Nghiên cứu đặt ra bốn câu hỏi: các toán tử tự cài có tuân thủ định nghĩa và quy ước biên không; Python và Fiji khác nhau thế nào khi nhận cùng đầu vào; hình thái và marker ảnh hưởng thế nào đến lỗi gộp hoặc chia; chi phí tính toán thay đổi ra sao giữa các cách cài đặt.

Phạm vi gồm đủ tám nội dung trong đề tài: erosion, dilation, opening, closing, morphological reconstruction, distance transform, watershed và connected components. Báo cáo trình bày từng phép và cách phối hợp trong quy trình hoàn chỉnh. Phân đoạn dựa trên độ sáng và hình dạng không gán tên ngữ nghĩa như người, xe hoặc động vật.



Ví dụ của dự án: ảnh tổng hợp có bảy vật thể, gồm hai cặp chạm nhau. Hình này minh họa đầu vào, kết quả Python và kết quả Fiji; không thay thế đánh giá trên ảnh thật.

Biểu diễn ảnh và các quy ước toán học

Với ảnh nhị phân, A là tập các tọa độ pixel tiền cảnh. Phần tử cấu trúc B là tập độ lệch quanh một gốc xác định. Chương trình dùng phần tử có kích thước lẻ 2r+1 để gốc nằm ở tâm. Ba hình dạng khảo sát là đĩa, vuông và chữ thập. Đĩa bán kính 1 trên lưới số nguyên có dạng chữ thập, khác hình vuông 3×3.

Lân cận 4 gồm trên, dưới, trái, phải. Lân cận 8 bổ sung bốn đường chéo. Đây là tham số có ảnh hưởng đến số vùng, khả năng lan tái tạo và đường phân chia Watershed. Không được đổi liên thông giữa hai bản rồi xem chênh lệch là lỗi triển khai.

$$N4(x,y) = \{(x-1,y),(x+1,y),(x,y-1),(x,y+1)\}$$

$$N8(x,y) = N4(x,y) \cup \{(x\pm1,y\pm1)\}$$

Ngoài miền ảnh, phép hình thái nhị phân coi pixel là nền 0. Distance transform thêm một khung nền dày một pixel; quy ước này quyết định khoảng cách khi toàn ảnh là tiền cảnh. Trong tái tạo ảnh xám, giá trị ngoài miền là âm vô cùng khi tính giãn, để không tạo mức sáng mới ở biên.

Ảnh cường độ được chuẩn hóa min–max về [0,1]. Ảnh hằng được ánh xạ thành 0; đây là quy ước xác định, không suy diễn rằng một ảnh hằng chứa đối tượng. Ngưỡng thủ công sử dụng đơn vị đã chuẩn hóa, còn h trong bản dựa trên khoảng cách có đơn vị pixel. Diện tích được đo theo pixel² do thí nghiệm chưa sử dụng kích thước vật lý của pixel.

Erosion và Dilation

Erosion, hay phép co, giữ vị trí x nếu toàn bộ phần tử B khi đặt tại x nằm trong A. Phép này làm vùng tiền cảnh nhỏ lại, có thể loại điểm lẻ hoặc phá cầu nối mảnh. Dilation, hay phép giãn, bổ sung các vị trí có thể đạt được bằng cách cộng một điểm của A với một độ lệch trong B. Nó làm vùng mở rộng và có thể nối hai vật thể vốn gần nhau.

$$A \ominus B = \{x : x+B \subseteq A\}$$

$$A \oplus B = \{a+b : a \in A, b \in B\}$$

Ở dạng phép toán pixel, erosion là AND các ảnh dịch theo x+b; dilation là OR các ảnh dịch theo x−b. Dấu của độ lệch quan trọng với phần tử bất đối xứng. Nếu chỉ thử phần tử vuông hoặc đĩa đối xứng, lỗi phản xạ có thể không bị phát hiện. Bộ kiểm thử có thêm phần tử bất đối xứng để kiểm tra trường hợp này.

Cài đặt không duyệt từng cửa sổ bằng Python: với mỗi điểm khác 0 của SE, lấy một lát của ảnh đã đệm, rồi AND hoặc OR trên toàn mảng NumPy. Nếu $N=H\times W$ và k là số điểm của SE, số phép toán tỷ lệ $O(Nk)$, bộ nhớ phụ tỷ lệ $O(N)$ ngoài ảnh đệm. Đây là cách cài dễ đối chiếu với định nghĩa; SE lớn vẫn có thể tốn thời gian.

Ảnh xám có định nghĩa tổng quát bằng min/max trên lân cận với phần tử phẳng. Phần tự cài và đối chiếu ở dự án này dùng hình thái nhị phân trong quy trình phân đoạn; tái tạo ảnh xám được xử lý riêng ở chương sau.

Opening và Closing

Opening là erosion rồi dilation với cùng phần tử cấu trúc. Closing là dilation rồi erosion. Opening thường loại vùng nhỏ không chứa được phần tử cấu trúc; closing thường khép khe nhỏ và lỗ nhỏ. Hai phép không phải hai phép nghịch đảo, vì thông tin bị loại có thể không khôi phục được.

$$A \circ B = (A \ominus B) \oplus B$$

$$A \bullet B = (A \oplus B) \ominus B$$

Trên miền vô hạn với quy ước phù hợp, opening có tính phản mở rộng và closing có tính mở rộng; cả hai là phép lũy đẳng. Trên miền hữu hạn với nền ngoài ảnh bằng 0 và cắt lại sau từng bước, cần thận trọng khi kiểm tra các tính chất tại biên. Báo cáo vì vậy nêu rõ quy ước đệm thay vì giả định mọi thư viện đều xử lý giống nhau.

Ví dụ với khối vuông 3×3 nằm giữa ảnh 7×7 và SE vuông 3×3, co tạo một pixel trung tâm, giãn tạo khối 5×5, opening khôi phục khối 3×3 ban đầu. Nếu thêm một pixel rời xa khối, opening loại pixel đó. Nếu hai vật thể chỉ cách nhau một khe hẹp, closing có thể nối chúng và làm bài toán tách khó hơn.

Phần mềm có cả opening thường, closing và opening rồi closing. Mặc định của quy trình nghiên cứu là opening bằng reconstruction, nhằm loại vùng không còn hạt giống sau co nhưng khôi phục hình dạng của những vùng còn hạt giống. Không có lựa chọn nào được coi là tối ưu cho mọi ảnh; phép ablation đo tác dụng thực tế của từng bước.

Ví dụ tính tay và kết quả kiểm tra

Chọn A là khối 3×3 tại hàng 2 đến 4 và cột 2 đến 4 của ảnh 7×7, chỉ số bắt đầu từ 0. B là toàn bộ khối 3×3 có gốc tại tâm. Tại tâm (3,3), B nằm hoàn toàn trong A nên erosion bằng 1. Tại (2,3), B chạm hàng 1 là nền nên erosion bằng 0. Dilation tại (1,3) bằng 1 vì một điểm của B dịch đến A; tại (0,3) vẫn bằng 0.

A	Erosion	Dilation
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 1 1 1 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0

Các ma trận trên được tạo từ code đang nộp bằng scripts/academic_checks.py và lưu ở docs/evidence/academic_checks/hand_examples.json. Có thể kiểm tra từng ô theo định nghĩa trước khi chạy lại script. Tổng số pixel tiền cảnh tương ứng của ảnh gốc, co và giãn là 9, 1 và 25. Hai phép opening và closing trả lại khối 3×3 trong ví dụ ở giữa ảnh này.

Ví dụ nhỏ giúp kiểm tra quy ước tọa độ và dấu dịch. Nó không thay cho thí nghiệm ảnh thật vì không có nhiễu, độ sáng không đồng nhất hoặc hình dạng phức tạp. Trường hợp biên và SE bất đối xứng được kiểm tra thêm trong tests/test_algorithms.py.

Tái tạo hình thái

Tái tạo bằng giãn nhận ảnh đánh dấu M và ảnh giới hạn F, cùng kích thước và thỏa $M \leq F$. Bắt đầu từ $X0=M$, mỗi bước giãn X rồi lấy min với F. Dãy tăng đơn điệu nhưng không vượt F. Trên ảnh số hữu hạn, quá trình đạt điểm bất động. Với ảnh nhị phân, min tương đương giao tập.

$$X0=M;\ X(k+1)=\min(\delta(Xk),F);\ R\delta F(M)=X \text{ khi } X(k+1)=Xk$$

Phân biệt hai khái niệm marker: ảnh đánh dấu của reconstruction là cường độ ban đầu cho lan truyền; marker của Watershed là ảnh nhãn các vùng khởi tạo. Chúng có vai trò khác nhau dù cùng tên tiếng Anh. Mặt nạ reconstruction giới hạn giá trị cường độ, còn mask nhị phân của Watershed giới hạn miền được gán nhãn.

Opening by reconstruction lấy M là ảnh sau erosion và F là ảnh nhị phân trước co. Vùng nào mất hết hạt giống sẽ biến mất; vùng còn ít nhất một hạt giống sẽ lan lại bên trong F. Cách này có thể giữ hình dạng vùng tốt hơn opening thường, nhưng cũng khôi phục cầu nối khiến hai vật thể vẫn liên thông. Vì vậy cần Watershed ở bước sau nếu muốn tách vật thể chạm nhau.

Lấp lỗ bằng reconstruction thực hiện trên phần bù: dùng nền ở biên ảnh làm hạt giống, lan trong nền, sau đó lấy phần bù của phần nền tiếp cận được. Nền không nối tới biên trở thành lỗ được lấp. Lựa chọn liên thông ảnh hưởng đến việc một đường chéo mảnh có thông ra ngoài hay không.

Cài đặt reconstruction bằng hàng đợi

Một cách trực tiếp là giãn toàn ảnh lặp đến hội tụ. Nó dễ hiểu nhưng có thể quét lại nhiều pixel đã ổn định. Bản tự cài dùng deque và cờ pending: khi một pixel tăng giá trị, chỉ cần lan ảnh hưởng tới các láng giềng. Cờ pending tránh đưa trùng một pixel vào hàng đợi khi nó đang chờ xử lý.

Trước vòng lặp, chương trình giãn đồng bộ một lượt để tính kết quả ban đầu, rồi chỉ đưa các vị trí tăng so với seed vào hàng đợi. Các vị trí không đổi đã truyền giá trị gốc cho láng giềng trong lượt đồng bộ này. Khi lấy p khỏi hàng đợi, ứng viên của q là min(X[p], F[q]); nếu lớn hơn X[q], cập nhật q và đưa q vào hàng đợi.

```
def reconstruction_dilation(seed, mask, connectivity=4):
    """Geodesic dilation to fixed point using a propagation queue (gray/binary)."""
    seed, mask = np.asarray(seed, float), np.asarray(mask, float)
    if seed.shape != mask.shape or seed.ndim != 2 or not np.isfinite(seed).all() or not np.isfinite(mask).all():
        raise ValueError("Seed and mask must be finite 2D arrays of equal shape")
    if np.any(seed > mask):
        raise ValueError("Dilation reconstruction requires seed <= mask")
    neighbors = offsets(connectivity)
    h,w=seed.shape
    padded=np.pad(seed,1,constant_values=-np.inf)
    result=seed.copy()
    for dy,dx in neighbors:
        np.maximum(result,padded[1+dy:1+dy+h,1+dx:1+dx+w],out=result)
    np.minimum(result,mask,out=result)
    h, w = result.shape
    pending = result > seed
    q = deque(zip(*np.nonzero(pending)))
    steps=0
    while q:
        steps+=1
        if steps%2048==0: check_cancelled()
        y, x = q.popleft()
        pending[y,x] = False
        value = result[y,x]
        for dy, dx in neighbors:
            ny, nx = y+dy, x+dx
            if 0 <= ny < h and 0 <= nx < w:
                candidate = min(value, mask[ny,nx])
                if candidate > result[ny,nx]:
                    result[ny,nx] = candidate
                    if not pending[ny,nx]:
                        pending[ny,nx] = True
                        q.append((ny,nx))
    return result
```

Hàm kiểm tra cùng kích thước, dữ liệu hữu hạn và seed≤mask. Các phép thử ảnh xám ngẫu nhiên đối chiếu với scikit-image trên liên thông 4 và 8. Tính đúng được xác nhận bằng đầu ra, không suy luận chỉ từ việc chương trình chạy xong. Hiệu quả tốc độ còn phụ thuộc số lần cập nhật và chi phí vòng lặp Python; phần thực nghiệm tối ưu trình bày phép đo thực tế.

Gán nhãn các vùng liên thông

Connected components labeling, viết tắt CCL, gán cùng một nhãn cho những pixel tiền cảnh có đường đi qua các pixel tiền cảnh theo liên thông đã chọn. Chương trình quét các pixel tiền cảnh; khi gặp điểm chưa có nhãn, tạo nhãn mới và duyệt BFS bằng hàng đợi để lan trong vùng đó.

$$L(p)=L(q) \text{ nếu p và q thuộc cùng thành phần liên thông của mask}$$

Mỗi pixel được gán nhãn tối đa một lần, mỗi điểm kiểm tra tối đa 4 hoặc 8 láng giềng. Độ phức tạp O(N), bộ nhớ O(N) cho ảnh nhãn và hàng đợi. Giá trị nhãn chỉ là mã định danh: kết quả [1,2] và [8,3] có thể biểu diễn cùng một phân đoạn. Không được so sánh trực tiếp màu tô hoặc ID để kết luận khác biệt thuật toán.

Ví dụ ảnh đường chéo 3×3 có ba pixel 1: liên thông 4 tạo ba vùng, liên thông 8 tạo một vùng. Ví dụ ảnh tổng hợp bảy vật thể có hai cặp chạm nhau: CCL có thể chỉ trả về năm vùng vì mỗi cặp nối nhau thành một thành phần.

Trong pipeline, CCL dùng để loại vùng dưới diện tích tối thiểu, tìm vùng chưa có marker và gán nhãn cho các cực đại. Nó cũng là baseline độc lập để đánh giá xem Watershed có thực sự cải thiện kết quả hay đang chia quá nhiều. Khi dữ liệu có nhiều vật thể đã tách rời, baseline đơn giản này có thể đạt F1 cao.

```
def connected_components(mask, connectivity=4):
    a = np.asarray(mask, bool)
    if a.ndim != 2:
        raise ValueError("Expected 2D mask")
    neighbors = offsets(connectivity)
    out = np.zeros(a.shape, np.int32)
    h, w = a.shape
    label = 0
    visited=0
    for y, x in zip(*np.nonzero(a)):
        if x == 0 or (int(y)*w+int(x))%2048 == 0: check_cancelled()
        if out[y, x]:
            continue
        label += 1
        out[y, x] = label
        q = deque([(int(y), int(x))])
        while q:
            visited+=1
            if visited%2048 == 0: check_cancelled()
            cy, cx = q.popleft()
            for dy, dx in neighbors:
                ny, nx = cy+dy, cx+dx
                if 0 <= ny < h and 0 <= nx < w and a[ny,nx] and out[ny,nx] == 0:
                    out[ny,nx] = label
                    q.append((ny,nx))
    return out
```

Biến đổi khoảng cách Euclid

Với mask A, distance transform trả khoảng cách từ mỗi điểm tiền cảnh tới nền gần nhất. Pixel nền có khoảng cách 0. Ở vật thể gần tròn, các điểm gần tâm thường có khoảng cách lớn; do đó các cực đại của ảnh khoảng cách là ứng viên marker.

$$D(p)=\min\{\|p-q\|_2 : q\notin A\}$$

$$D^2(x,y)=\min\{(x-u)^2+(y-v)^2 : (u,v) \text{ là nền}\}$$

Tính khoảng cách tới mọi pixel nền một cách trực tiếp có chi phí lớn. Bản tự cài sử dụng thuật toán tách trục của Felzenszwalb và Huttenlocher [3]: đặt $f=0$ ở nền và $f=+\infty$ ở tiền cảnh, tính biến đổi một chiều theo cột rồi theo hàng. Với mỗi chiều, tìm bao dưới của các parabol $(x-i)^2+f(i)$. Các điểm giao quyết định khoảng chỉ số mà mỗi parabol nhỏ nhất.

Mỗi parabol được thêm và loại khỏi bao hữu hạn lần, nên biến đổi một chiều có chi phí tuyến tính theo độ dài. Hai lượt toàn ảnh cho $O(HW)$, rồi lấy căn bậc hai. Triển khai bỏ qua các vị trí có chi phí vô cùng và thêm khung nền một pixel để trường hợp toàn tiền cảnh vẫn hữu hạn.

Khoảng cách Euclid chính xác, chamfer và ImageJ EDM có thể có sai khác. Khi đối chiếu riêng Watershed, dự án dùng chung ảnh elevation float32 thay vì giả định các hàm distance transform của hai nền tảng giống hệt nhau. Khi chạy Fiji độc lập, khác biệt về distance map được ghi là một phần của khác biệt quy trình.

Tính khoảng cách bằng bao dưới của parabol

Trong một chiều, mỗi vị trí nền sinh một parabol bình phương khoảng cách. Hai vị trí i và j có điểm giao s bằng công thức bên dưới. Mảng v giữ các chỉ số parabol còn hữu ích; mảng z giữ mốc giao. Khi một parabol mới làm mốc giao lùi trước miền hiệu lực của parabol cuối, loại parabol cuối khỏi bao.

$$s = [(f(j)+j^2)-(f(i)+i^2)] / [2(j-i)]$$

```
def _squared_edt_1d(f):
    """Lower parabola envelope, O(n); Felzenszwalb & Huttenlocher (2012).

    Independently written Python implementation of the published algorithm;
    computes min_i ((x-i)^2 + f[i]), skipping infinite-cost sites.
    """
    n = len(f)
    sites = np.flatnonzero(np.isfinite(f))
    if not sites.size:
        return np.full(n, np.inf)
    v = np.empty(n, int)
    z = np.empty(n+1, float)
    k = 0
    v[0] = sites[0]
    z[0], z[1] = -np.inf, np.inf
    for q in sites[1:]:
        s = ((f[q] + q*q) - (f[v[k]] + v[k]*v[k])) / (2.0*(q-v[k]))
        while s <= z[k]:
            k -= 1
            s = ((f[q]+q*q)-(f[v[k]]+v[k]*v[k]))/(2.0*(q-v[k]))
        k += 1
        v[k], z[k], z[k+1] = q, s, np.inf
    result = np.empty(n, float)
    k = 0
    for q in range(n):
        while z[k+1] < q:
            k += 1
        result[q] = (q-v[k])**2 + f[v[k]]
    return result
```

Sau khi có bao, quét x tăng dần để xác định parabol hiệu lực rồi tính giá trị. Hai lượt hàng và cột dùng chung hàm. Kiểm thử so sánh với scipy.ndimage.distance_transform_edt trên ảnh ngẫu nhiên, ảnh rỗng và ảnh toàn tiền cảnh với cùng quy ước khung nền.

Với khối vuông 3×3 đặt giữa nền, distance map trong khối là [1,1,1; 1,2,1; 1,1,1]. Cực đại tại tâm có độ cao 2. Kết quả này giải thích trực tiếp vì sao khoảng cách có thể xác định vùng sâu bên trong vật thể, nhưng một vật thể dài hoặc không lồi vẫn có thể sinh nhiều cực đại.

Marker và lọc cực đại theo h

Nếu lấy mọi cực đại địa phương làm marker, nhiễu hoặc gồ ghề nhỏ trên distance map có thể tạo quá nhiều hạt giống. Watershed sẽ chia một đối tượng thật thành nhiều mảnh. H-maxima giảm ảnh hưởng của cực đại có độ nổi thấp bằng reconstruction ảnh xám.

$$R=R_{\delta D}(D-h);\ H_h=\{p : D(p)-R(p)\geq h\}$$

Trong code, ngưỡng so sánh có dung sai 10^{-9} để hạn chế sai số dấu phẩy động. Ảnh kết quả cực đại được giao với mask và gán nhãn liên thông. Nếu một thành phần tiền cảnh không có marker, chương trình bổ sung một marker tại vị trí có khoảng cách lớn nhất trong thành phần đó. Quy tắc dự phòng ngăn vật thể bị bỏ trắng khi h quá cao, nhưng có thể gộp toàn thành phần vào một vật thể.

Bốn phương án được so sánh là CCL, raw marker, hmax và adaptive. Raw lấy cực đại chưa lọc. Hmax dùng h cố định. Adaptive sử dụng h riêng cho thành phần: max(h, α×Dmax). Đây là một giả thuyết cải tiến để thích ứng kích thước, chưa phải bảo đảm tăng độ chính xác.

Tăng h thường giảm marker và giảm chia thừa, nhưng tăng nguy cơ gộp vật thể. Giảm h có thể tách được nhiều tâm sát nhau nhưng nhạy với nhiễu. Khảo sát chọn h phải dùng validation; sau khi xem test không được tiếp tục sửa h rồi coi số liệu test mới là đánh giá độc lập.

Watershed có điều khiển marker

Watershed xem ảnh cường độ như địa hình và lan các nhãn từ điểm khởi tạo. Trong bài toán này, elevation E=−D khiến các vùng sâu trong vật thể trở thành vùng thấp. Mask ngăn nhãn lan ra nền. Mỗi marker mang một nhãn, các pixel chưa gán nhận nhãn trong quá trình lan.

Hàng đợi ưu tiên của bản tự cài sắp theo cặp (mức ngập, thứ tự đến). Khi mở rộng từ p tới q, mức ngập mới bằng max(mức hiện tại, E[q]). Các pixel marker giữ nguyên nhãn. Pixel được đưa vào hàng đợi khi gặp lần đầu; thứ tự đến quyết định tranh chấp ở vùng có cùng mức ưu tiên. Bản này không tạo đường phân cách nền, gọi là không tạo dams.

$$\text{cost}(q)=\max(\text{cost}(p),E(q))$$

Quy tắc này tạo đầu ra xác định với thứ tự duyệt và liên thông cố định. Tuy nhiên một thư viện khác có thể xử lý plateau, thứ tự khởi tạo, khoảng cách trên vùng phẳng hoặc đường phân cách khác. Đó là lý do Fiji shared có thể khác ranh giới dù cùng mask và marker. Báo cáo không dùng tuyên bố tương đương tuyệt đối giữa mọi biến thể Watershed.

Với heap nhị phân và mỗi pixel được đưa vào tối đa một lần, chi phí chính O(N log N), bộ nhớ O(N). Tính đúng cơ bản được kiểm tra bằng các bất biến: nhãn ngoài mask bằng 0; nhãn marker được bảo toàn; thành phần có marker nhận nhãn. Chất lượng so với đối tượng thật cần đánh giá bằng ground truth riêng.

Mã cài đặt Watershed

Mã dưới đây là phần tự cài trong morpholab/algorithms.py. Nó sử dụng heapq của Python làm cấu trúc dữ liệu hàng đợi, không gọi hàm watershed của thư viện để tạo kết quả custom. Các cấu trúc deque và heapq là tiện ích lập trình, cần phân biệt với việc sử dụng sẵn toàn bộ thuật toán phân đoạn.

```
def watershed(elevation, markers, mask, connectivity=4):
    """Seeded priority flood without dams.

    Heap key = (maximum elevation along path, FIFO arrival). Marked pixels are
    immutable. Equal-cost plateaus are assigned by arrival order. This tie rule
    is documented; exact boundaries need not match another implementation.
    """
    a = np.asarray(elevation, float)
    marker = np.asarray(markers)
    valid = np.asarray(mask, bool)
    if a.shape != marker.shape or a.shape != valid.shape or a.ndim != 2:
        raise ValueError("Elevation, markers and mask must have equal 2D shapes")
    if not np.isfinite(a).all() or np.any(marker < 0) or np.any(marker != np.floor(marker)):
        raise ValueError("Finite elevation and nonnegative integer markers required")
    if np.any((marker > 0) & ~valid):
        raise ValueError("Markers must be inside the mask")
    out = marker.astype(np.int32).copy()
    queued = out > 0
    heap = []
    age = 0
    for y, x in zip(*np.nonzero(out)):
        heapq.heappush(heap, (float(a[y,x]), age, int(y), int(x)))
        age += 1
    neighbors = offsets(connectivity)
    h, w = a.shape
    steps=0
    while heap:
        steps+=1
        if steps%2048==0: check_cancelled()
        cost, _, y, x = heapq.heappop(heap)
        for dy, dx in neighbors:
            ny, nx = y+dy, x+dx
            if 0 <= ny < h and 0 <= nx < w and valid[ny,nx] and not queued[ny,nx]:
                queued[ny,nx] = True
                out[ny,nx] = out[y,x]
                heapq.heappush(heap, (max(cost,float(a[ny,nx])), age, ny, nx))
                age += 1
    return out
```

Thí nghiệm cần lưu chính elevation, markers, mask và connectivity mà hàm này nhận. Một ảnh overlay đơn lẻ không đủ để tái lập, vì thay đổi tiền xử lý hoặc marker có thể thay đổi mạnh số vùng và đường biên. Dữ liệu xuất của dự án chứa các TIFF trung gian và run.json để đáp ứng yêu cầu này.

Quy trình Python hoàn chỉnh

Ảnh được đọc, kiểm tra kích thước và chuẩn hóa. Gaussian sigma=1 làm trơn trước Otsu mặc định. Tiền cảnh sáng sử dụng điều kiện I>T; nếu đối tượng tối dùng I<T. Ngưỡng −1 trong cấu hình yêu cầu Otsu, còn [0,1] là ngưỡng thủ công.

Tiếp theo là hình thái theo operation, lấp lỗ nếu bật và loại thành phần có diện tích nhỏ hơn min_area. Gán lại nhãn thành phần, tính khoảng cách, tạo marker theo mode và chạy Watershed trên âm khoảng cách. Sau cùng gán lại ID liên tiếp và xuất các giai đoạn để kiểm tra.

Tham số	Giá trị
engine	custom
mode	hmax
sigma	1.0
threshold	-1.0
bright_foreground	True
operation	opening_reconstruction
shape	disk
radius	1
min_area	25
fill_holes	True
connectivity	4
h	2.0
adaptive_fraction	0.15

Hai engine dùng cùng cấu hình nhưng có triển khai riêng: custom là các hàm tự viết, library dùng SciPy và scikit-image. Gaussian, Otsu, chuyển ảnh xám, đọc ghi, thống kê vùng và chuẩn hóa không được tuyên bố là tự cài. Việc nêu rõ ranh giới này giúp đánh giá đúng đóng góp lập trình của nhóm.

Thời gian total trong pipeline đo phần xử lý, loại đọc ghi và tính metric bên ngoài. Một số ảnh trung gian như erosion và dilation vẫn được tính để phục vụ nghiên cứu dù không được dùng bởi một mode cụ thể; khi so tốc độ phải giữ cùng phạm vi công việc. Không dùng số này như tốc độ tối thiểu của một chương trình production đã loại bỏ mọi bước minh họa.

Fiji và plugin MorphoLibJ

Fiji là bản phân phối ImageJ kèm hệ sinh thái plugin. MorphoLibJ cung cấp các phép hình thái, tái tạo, gán nhãn và Watershed để khảo sát và tham chiếu [1,2]. Máy thực nghiệm sử dụng ImageJ 1.54p và MorphoLibJ 1.6.5. Plugin được bật từ update site IJPB-plugins và xuất hiện trong Plugins > MorphoLibJ.

Đề tài sử dụng hai cách tham chiếu. Fiji shared chỉ kiểm tra bước Watershed khi nhận cùng elevation, markers, mask và liên thông. Fiji native tự thực hiện một quy trình tiền xử lý và phân đoạn bằng ImageJ/MorphoLibJ. Hai cách trả lời hai câu hỏi khác nhau: độ nhất quán triển khai và hiệu quả của một quy trình tham chiếu độc lập.

Adapter Java gọi API thật của MorphoLibJ, không thay kết quả Fiji bằng kết quả Python. Fiji nhận TIFF có kiểu dữ liệu thích hợp và trả ảnh nhãn. MorphoBridge còn tính IoU, Dice tiền cảnh và số đối tượng bằng Java độc lập với bộ metric Python. Điều này tạo bằng chứng định lượng từ cả hai phía.

Các lớp Java được biên dịch với JAR trong thư mục Fiji đã cài. Python chỉ điều phối tiến trình, lưu tham số và thu dữ liệu. Khi muốn minh họa trực tiếp trong Fiji, dùng script Groovy đã xuất để mở ảnh và chạy lại bước Watershed trong Script Editor. Quy trình này có thể thực hiện mà không mở ứng dụng MorphoLab.

Thực hành kiểm chứng Watershed trong Fiji

Trước hết chạy lệnh Python cho ảnh mẫu với tùy chọn --fiji để tạo thư mục dữ liệu kiểm chứng. Mở Fiji tại C:/Tools/Fiji/fiji-windows-x64.exe. Chọn File > Open để mở elevation.tif, markers.tif và mask.tif trong thư mục kết quả Fiji. Giữ elevation dạng 32-bit, marker là nhãn số và mask là nhị phân.

Trong Plugins > MorphoLibJ > Segmentation, chọn Marker-controlled Watershed. Tên menu có thể được tìm bằng ô tìm lệnh của Fiji. Chọn ảnh địa hình làm input, ảnh marker làm marker và ảnh mask làm mask nếu hộp thoại cung cấp lựa chọn đó. Đặt liên thông 4 như cấu hình mẫu và tắt Calculate dams. Không thay bằng ảnh distance dương vì sẽ đảo ý nghĩa vùng thấp và vùng cao.

Cách tái lập ít phụ thuộc thao tác chọn cửa sổ là mở Run_in_Fiji.groovy trong Script Editor, chọn ngôn ngữ Groovy, bấm Run. Script mở đúng ba ảnh, gọi Watershed.computeWatershed, hiển thị và lưu fiji_interactive_labels.tif. Đây là script tự động theo gợi ý phân công, không cần viết thêm Macro cùng chức năng chỉ để có thêm một định dạng.

Sau khi chạy, lưu ảnh nhãn bằng TIFF. Có thể đổi LUT để xem các vùng, nhưng không lưu RGB thay thế ảnh nhãn dùng đánh giá. Chụp cửa sổ Fiji với ảnh gốc và ảnh nhãn để minh họa khi thuyết trình. Dùng lệnh evaluate hoặc chức năng nạp kết quả Fiji để tính lại chỉ số trên ảnh nhãn đã lưu.

```
#@ File(label="Thu muc ket qua Fiji chua elevation markers mask", style="directory") folder
#@ Integer(label="Lien thong (4 hoac 8)", value=4) connectivity
#@ Boolean(label="Hien anh ket qua", value=true) showImages
// Fiji Script Editor: Language > Groovy. Run, then choose outputs/demo_shared/fiji.
import ij.IJ
import inra.ijpb.watershed.Watershed
def dir = folder.getAbsolutePath()
if (!(connectivity in [4, 8])) throw new IllegalArgumentException("Connectivity must be 4 or 8")
def images = ["elevation", "markers", "mask"].collect { name ->
    def file = new File(folder, name + ".tif")
    if (!file.isFile()) throw new IllegalArgumentException("Missing " + file)
    IJ.openImage(file.getAbsolutePath())
}
if (showImages) images.each { it.show() }
def labels = Watershed.computeWatershed(images[0], images[1], images[2], connectivity, false, false)
labels.setTitle("MorphoLibJ labels nhom 1")
if (showImages) labels.show()
IJ.saveAsTiff(labels, new File(folder, "fiji_interactive_labels.tif").getAbsolutePath())
IJ.log("Saved Fiji labels, connectivity=" + connectivity + ", no dams: " + dir)
```

Quy trình Fiji độc lập và các khác biệt

NativePipeline.java đọc ảnh xám chuẩn hóa, lượng tử hóa 8-bit, chạy Gaussian của ImageJ và chọn ngưỡng Otsu từ histogram. Phần tử cấu trúc là SquareStrel với bán kính cấu hình. Fiji tự xuất erosion, dilation, opening, closing, reconstruction, mask, connected components, distance, marker, elevation và kết quả nhãn.

Mask mặc định dùng mở bằng reconstruction, lấp lỗ và lọc diện tích. Khoảng cách dùng ImageJ EDM. Marker dùng MorphoLibJ extended maxima, sau đó gán nhãn và bổ sung hạt giống cho thành phần chưa có marker. Watershed chạy không tạo dams. Quy ước chi tiết được lưu trong native_parameters.txt.

Bước	Python mặc định	Fiji native
Cường độ	float chuẩn hóa	Lượng tử hóa 8-bit
SE	Đĩa r=1	Vuông r=1
Khoảng cách	Euclid tách trục	ImageJ EDM
Cực đại	h-maxima tự cài/thư viện	MorphoLibJ extended maxima
Lấp lỗ	Liên thông đã chọn	Mặc định plugin
Đo thời gian	Không đọc ghi	Có ghi ảnh trung gian

Native là một baseline cố định, không sao chép mọi lựa chọn GUI của Python: shape, operation, mode, ngưỡng thủ công, fill_holes và marker thủ công không được chuyển thành cùng thao tác. Việc gọi nó “độc lập” không có nghĩa đầu vào cường độ thô hoặc mọi thuật toán hoàn toàn giống Python; ảnh gray chuẩn hóa là điểm chung, sau đó Fiji tự tính các bước.

Đo thời gian native bao gồm ghi ảnh TIFF trung gian. Thời gian Fiji shared riêng Watershed không bao gồm JVM; process_seconds bao gồm khởi động và adapter. Vì phạm vi đo khác nhau, không lấy trực tiếp các cột này chia nhau để công bố một speedup chung. Đây là kiểm soát quan trọng khi viết phần benchmark.

Dataset và nhãn đúng

Nghiên cứu sử dụng hai nguồn dữ liệu. Bộ tổng hợp gồm 18 ảnh 192×192 với seed 20261108, chia 6 validation và 12 test. Các đĩa có độ sáng, kích thước và vị trí biến thiên; vùng giao được phân cho tâm có khoảng cách chuẩn hóa nhỏ nhất. Nhãn hình học được tạo đồng thời với ảnh nên kiểm tra được lỗi tách. Đây là mô hình hóa đơn giản, không đại diện đầy đủ ảnh tế bào.

BBBC039v1 là bộ ảnh nhân tế bào U2OS huỳnh quang do Broad Bioimage Benchmark Collection cung cấp [4]. Dữ liệu gồm 200 ảnh TIFF 16-bit kích thước 520×696 và annotation PNG. Dự án giữ phân chia chính thức 100 training, 50 validation và 50 test. Các ảnh cùng nguồn thí nghiệm có thể còn tương quan; 200 ảnh không đồng nghĩa với 200 thí nghiệm sinh học độc lập.

Annotation PNG cần giải mã, không dùng trực tiếp màu như số nhãn. Dự án gán nhãn các vùng liên thông có cùng giá trị trên kênh đầu, nền 0, liên thông 8 theo ví dụ của tác giả [5]. Manifest lưu tên ảnh, split, đường dẫn ảnh và nhãn, cùng checksum của archive để kiểm tra nguồn.

Chưa huấn luyện mô hình nên training không được dùng để cập nhật trọng số. Validation chọn tham số và khảo sát phương án. Test dùng báo cáo cuối trên cấu hình đã cố định. Nếu phát triển tiếp sau khi xem test, phải công khai rằng tập test này đã tham gia vòng phát triển và cần dữ liệu đánh giá độc lập mới để xác nhận cải tiến.

Chỉ số định lượng và cách ghép đối tượng

IoU và Dice tiền cảnh đánh giá mức trùng nhau của hai mặt nạ, không phân biệt các nhãn bên trong. F1 theo đối tượng cần ghép nhãn dự đoán với nhãn đúng. Dự án chỉ cho ghép cặp có IoU≥0,5; mỗi nhãn dùng tối đa một lần. Hungarian tối đa số cặp hợp lệ trước, rồi tối đa tổng IoU trong các cặp đó.

$$\mathrm{IoU}=|P\cap G|/|P\cup G|;\ \mathrm{Dice}=2|P\cap G|/(|P|+|G|)$$

$$\mathrm{Precision}=TP/(TP+FP);\ \mathrm{Recall}=TP/(TP+FN);\ F1=2TP/(2TP+FP+FN)$$

$$PQ=\Sigma\ \mathrm{IoU}(\text{cặp đúng})/(TP+0{,}5FP+0{,}5FN)$$

TP là số cặp ghép đúng; FP là dự đoán chưa ghép; FN là vật thể thật bị bỏ sót. Sai số đếm tuyệt đối là $|n_{pred}-n_{true}|$. Khi cả hai ảnh rỗng, IoU/Dice/F1 quy ước bằng 1; nếu chỉ một ảnh rỗng, F1 bằng 0. Quy ước này được kiểm thử và phải đọc cùng số vật thể, tránh hiểu "ảnh không có gì đạt 100%" là kết quả phân đoạn tốt trên ảnh có đối tượng.

Split/merge dùng một quan hệ chẩn đoán: hai vùng có giao ít nhất 10% diện tích vùng nhỏ hơn. Một nhãn thật liên hệ với nhiều dự đoán là split; một dự đoán liên hệ với nhiều nhãn thật là merge. Đây không phải số lỗi tuyệt đối duy nhất theo mọi chuẩn đánh giá, mà là quy ước minh bạch của dự án.

So Fiji với Python sử dụng phép ghép ID riêng theo số pixel giao để đo đồng thuận trên hợp tiền cảnh. Đồng thuận cao chỉ cho thấy hai triển khai gần nhau. Muốn nói đúng với vật thể thật vẫn phải dùng ground truth.

Thiết kế thí nghiệm và kiểm soát sai lệch

Thí nghiệm so sánh bốn cách chạy trên cùng 50 ảnh test: Python custom, Python library, Fiji shared và Fiji native. Cấu hình Python cố định hmax, sigma=1, Otsu, SE đĩa bán kính 1, min_area=25, h=2, liên thông 4, đối tượng sáng và lấp lỗ. Fiji native sử dụng quy ước riêng đã nêu. Mỗi hàng per_image.csv là một cặp ảnh và cách chạy.

Khảo sát h ban đầu chạy library/hmax trên 6 ảnh validation đầu với h□{0,5;1;2;4;6}. Kết quả chọn h=2. Từ bản 1.2, chức năng sweep tôn trọng engine đang chọn và xuất đúng mode hmax. Khi tái lập số liệu cũ, dùng cấu hình library; không thay engine rồi cho rằng đó là cùng một thí nghiệm.

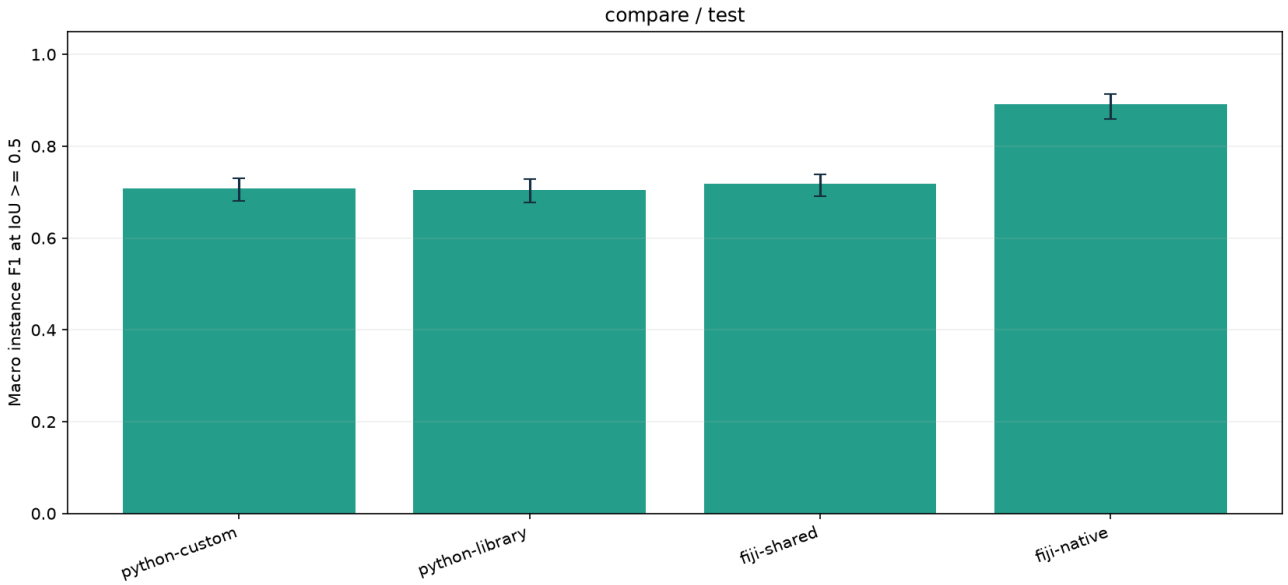
Phép ablation gồm baseline, bỏ Gaussian, bỏ hình thái, opening thường, bỏ lấp lỗ, marker raw và CCL. Khảo sát độ bền gồm ảnh gốc, nhiễu Gaussian σ=0,02/0,05/0,1 và làm mờ σ=1/2/3. Mỗi khảo sát dùng 6 validation, tổng 42 dòng. Các ảnh biến đổi cùng nguồn không được tính như các mẫu độc lập mới.

Python có một lượt warm-up trước đo. Đợt so sánh đã lưu dùng một lượt đo mỗi cấu hình mỗi ảnh; do đó số thời gian là mô tả, chưa đủ kết luận ổn định. Công cụ cho phép tăng repeats và lấy trung vị. Chất lượng lấy trung bình F1 theo ảnh, không gộp mọi vật thể của cả tập thành một ảnh lớn.

CI 95% được tính bằng bootstrap percentile 1000 lần lấy mẫu ảnh với seed cố định. CI thể hiện biến thiên trong tập khảo sát; các khoảng chồng lấn hoặc không chồng lấn không tự thay cho kiểm định cặp hay nghiên cứu trên dữ liệu độc lập.

Kết quả đối chiếu trên 50 ảnh test

Cách chạy	Ảnh	F1	CI 95%	IoU	Sai số đếm
python-custom	50	0.7087	0.6807–0.7305	0.8954	21.10
python-library	50	0.7054	0.6771–0.7277	0.8954	21.10
fiji-shared	50	0.7188	0.6914–0.7386	0.8954	21.10
fiji-native	50	0.8917	0.8592–0.9132	0.8910	13.58



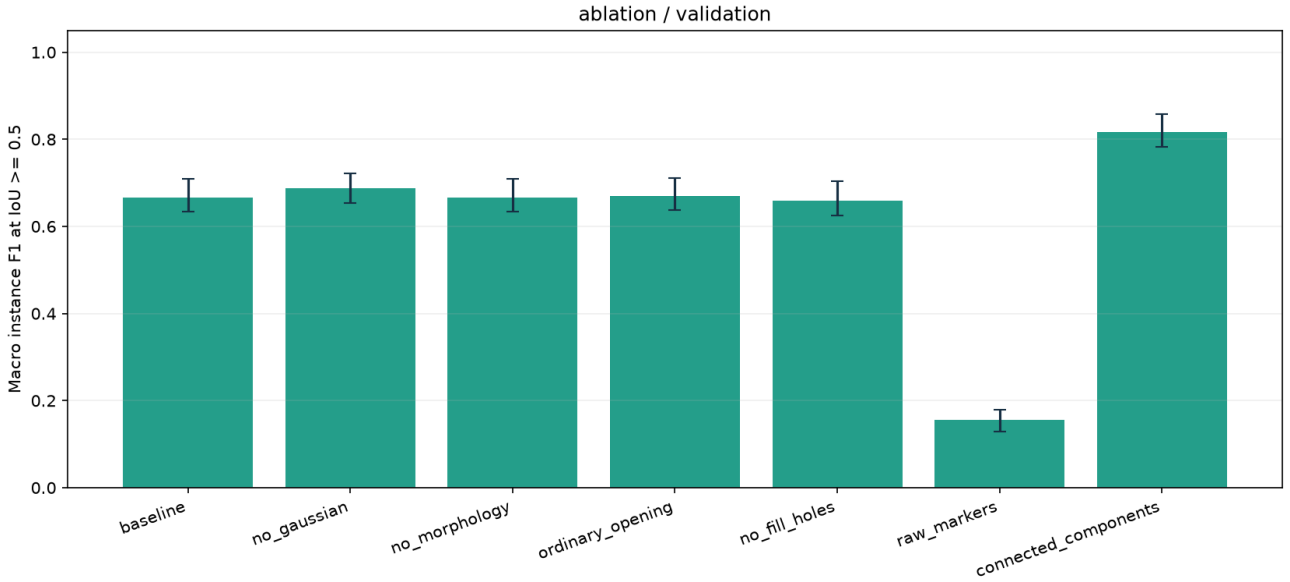
Python custom, Python library và Fiji shared có cùng IoU tiền cảnh khoảng 0,8954 và sai số đếm trung bình 21,10, nhưng F1 khác nhau. Điều này phù hợp với việc chúng chia cùng mask thành cùng số nhãn nhưng đường biên giữa các nhãn không hoàn toàn giống nhau. Trung vị đồng thuận pixel của shared so với custom khoảng 95,56%.

Fiji native đạt F1 0,8917 trên tập này, đồng thời IoU tiền cảnh 0,8910 thấp hơn nhẹ. Kết quả cho thấy chất lượng mask và chất lượng tách đối tượng là hai tiêu chí khác nhau. Native có ít split hơn rõ rệt nhưng vẫn có merge; không có cơ sở kết luận mọi ảnh hoặc mọi miền dữ liệu đều tốt hơn.

Dữ liệu nguồn gồm 200 dòng không lỗi. CSV và study.json được lưu tại docs/evidence/v2_compare_bbbc039_test. Biểu đồ và bảng trong báo cáo được lấy từ dữ liệu đã đo, không lấy số từ ảnh chụp GUI hay kết quả công bố của phương pháp khác.

Phân tích ảnh hưởng của từng bước

Biến thể	F1	IoU	Sai số đếm
baseline	0.6663	0.8798	24.5000
no_gaussian	0.6884	0.8904	22.3333
no_morphology	0.6663	0.8798	24.5000
ordinary_opening	0.6692	0.8787	24.3333
no_fill_holes	0.6592	0.8791	25.3333
raw_markers	0.1563	0.8798	298.5000
connected_components	0.8168	0.8798	16.3333



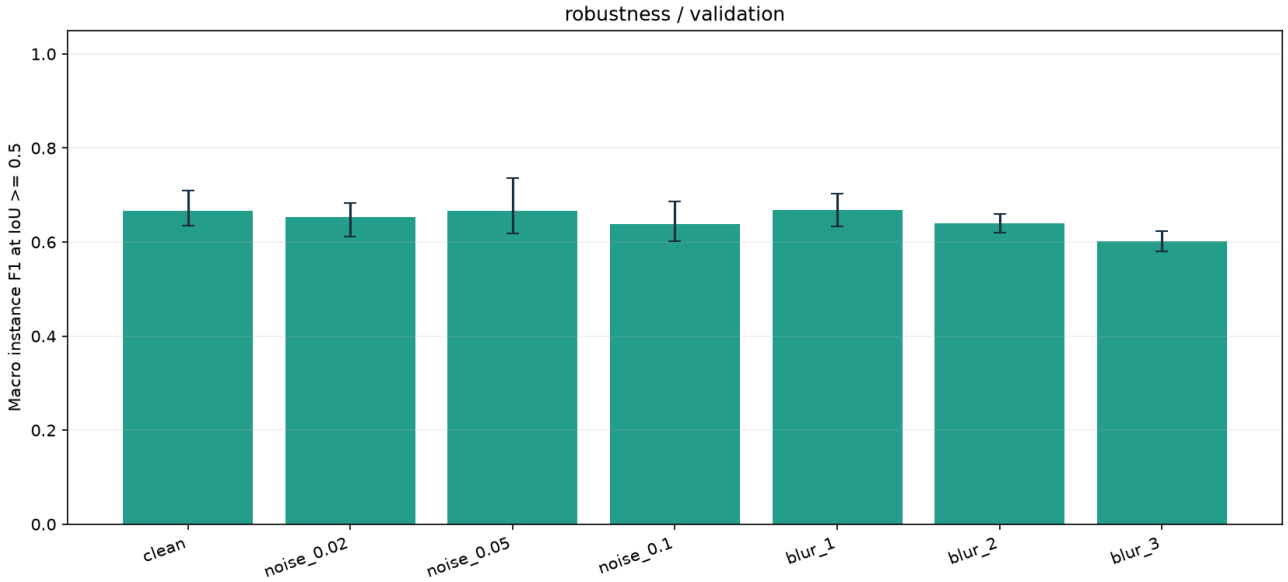
Trên sáu ảnh validation này, bỏ Gaussian tăng F1 từ 0,6663 lên 0,6884; bỏ hình thái không thay đổi F1 ở cấu hình hiện tại. Marker raw làm F1 giảm mạnh còn 0,1563 và tạo nhiều split. CCL đạt 0,8168, cao hơn baseline Watershed, cho thấy marker ở baseline vẫn có xu hướng chia thừa trên các ảnh khảo sát.

Các kết quả không có nghĩa Gaussian hoặc reconstruction luôn vô ích. Tác dụng phụ thuộc độ nhiễu, kích thước SE, cấu trúc vật thể và cách chọn marker. Phép thử chỉ thay một yếu tố trong cấu hình hiện tại, chưa khảo sát tương tác giữa nhiều tham số. Việc giữ một bước trong quy trình cần được giải thích bằng giả thuyết và số liệu, không dựa vào số lượng bước càng nhiều càng tốt.

Một hướng phát triển hợp lý là chọn marker ổn định hơn và khảo sát h cùng sigma trên validation. Đề xuất này chưa được chứng minh là tăng điểm trên dữ liệu độc lập; báo cáo không nâng nó thành kết luận thực nghiệm.

Ảnh hưởng của nhiễu và làm mờ

Biến thể	F1	IoU	Sai số đếm
clean	0.6663	0.8798	24.5000
noise_0.02	0.6527	0.8736	23.6667
noise_0.05	0.6668	0.8374	18.8333
noise_0.1	0.6385	0.7792	18.0000
blur_1	0.6689	0.8751	21.6667
blur_2	0.6406	0.8616	19.3333
blur_3	0.6018	0.8461	20.3333



F1 của ảnh gốc khoảng 0,6663; khi làm mờ sigma=3 giảm còn 0,6018. IoU tiền cảnh cũng giảm, cho thấy mất chi tiết biên ảnh hưởng đến cả mask và marker. Nhiễu nhẹ không tạo xu hướng đơn điệu tuyệt đối của F1 trên tập nhỏ, vì một thay đổi cường độ có thể làm số marker tăng hoặc giảm theo những cách khác nhau.

Nhiễu được thêm sau chuẩn hóa và clip về [0,1]. Làm mờ giữ nguyên nhãn đúng để đo sức chịu đựng của thuật toán trước suy giảm ảnh; không tạo nhãn mới thích ứng với ảnh đã mờ. Đây là stress test có kiểm soát, không phải mô phỏng đầy đủ mọi nguồn nhiễu của kính hiển vi.

Kết quả cần đọc cùng ảnh trung gian. Nếu mask đã mất một vùng, Watershed không thể tạo lại vùng đó từ nền bị loại. Nếu mask vẫn tốt nhưng marker quá dày, lỗi chủ yếu nằm ở bước chia đối tượng. Việc lưu từng giai đoạn giúp phân biệt hai nguyên nhân này và đề xuất sửa đúng chỗ.

Benchmark và thực nghiệm tối ưu

Cạnh ảnh	Cách cài	Trung vị s	Min s	Max s
48	queue_python	0.008983	0.008805	0.009617
48	synchronous_scipy	0.002225	0.002203	0.002262
96	queue_python	0.035766	0.035478	0.036634
96	synchronous_scipy	0.009409	0.008918	0.011008
160	queue_python	0.100107	0.098277	0.108437
160	synchronous_scipy	0.030336	0.028342	0.031392

Thí nghiệm bổ sung tạo mask hằng 1 kích thước 48, 96 và 160, với một seed sáng tại tâm, liên thông 4. So sánh reconstruction giãn đồng bộ dùng SciPy với bản hàng đợi Python. Mỗi trường hợp warm-up một lần, đo năm lần, kiểm tra đầu ra giống hệt và loại thời gian đọc ghi khỏi phép đo. Kết quả đầy đủ nằm trong timing.csv và protocol.json.

Bản queue Python chậm hơn bản synchronous dùng SciPy trên cả ba kích thước được thử. Đây là kết quả thực, không phải lỗi cần che đi: số phép cập nhật lý thuyết ít hơn chưa bảo đảm thời gian nhỏ hơn khi một bên dùng vòng lặp Python và một bên gọi nhân xử lý đã biên dịch. Thử nghiệm này so hai triển khai thực tế, không cô lập riêng độ phức tạp thuật toán.

Trong so sánh 50 ảnh, trung vị thời gian pipeline custom khoảng 6,9673 giây và library khoảng 0,3184 giây. Hai bản có cùng mục tiêu đầu ra nhưng chi phí vòng lặp khác. Native gồm ghi ảnh trung gian, shared chỉ đo một bước, nên không đặt tất cả vào cùng phép chia tốc độ. Không dùng đợt đo này để kết luận phần cứng khác sẽ có cùng thời gian.

Cải tiến đã cài gồm dùng lát NumPy cho hình thái, deque có cờ pending cho reconstruction và EDT tách trục tuyến tính. Mỗi cải tiến cần giữ đầu ra đúng qua kiểm thử. Hướng tối ưu tiếp theo là biên dịch vòng lặp nóng hoặc giảm xuất trung gian trong chế độ chạy nhanh, đồng thời đo lại có kiểm soát.

Kiểm thử tính đúng và khả năng tái lập

Bộ kiểm thử có 29 trường hợp chạy qua trên môi trường đã ghi, gồm thuật toán, metric, Java/Fiji, phiên làm việc và bảy luồng giao diện thực. Các test không chỉ so ảnh bằng mắt: morphology đối chiếu SciPy cả SE bất đối xứng; reconstruction so ảnh xám ngẫu nhiên; EDT kiểm tra biên; CCL thử liên thông; Watershed kiểm tra mask và marker; metric thử hoán vị ID, ảnh rỗng, lỗi split và merge.

FijiIntegrationTest gọi plugin thật và đối chiếu chỉ số Java với Python. Kiểm thử phiên lưu xác nhận ảnh, cấu hình và nhãn được khôi phục. Kiểm thử hủy xác nhận worker dừng và tiến trình con được thu hồi. Biên bản chi tiết nằm trong docs/KIEM_THU_PHAN_MEM.md; số test qua không đồng nghĩa mọi ảnh sẽ phân đoạn đúng.

```
conda activate ai
cd /d D:\XU_LY_ANH
python -B -m unittest discover -s tests -v
python -B scripts/academic_checks.py

REM Chay them 7 test Tk tren Windows
set MORPHOLAB_DESKTOP_TEST=1
python -B -m unittest discover -s tests -v
set MORPHOLAB_DESKTOP_TEST=
```

Một phép thử quan trọng cho người dùng là đặt min_area=100000 trên ảnh 192×192. Toàn ảnh chỉ có 36864 pixel nên mask sau lọc phải rỗng. Với ground truth có vật thể, F1 phải bằng 0; nếu không có ground truth, F1 phải để trống thay vì tự gán một điểm. Đây là kiểm thử hành vi và cách diễn giải, không phải một cấu hình hợp lý để phân đoạn.

Để tái lập, giữ manifest, file cấu hình, phiên bản thư viện, loại dữ liệu và phạm vi tính thời gian. Lệnh có cùng seed tạo cùng ảnh tổng hợp; số thời gian vẫn biến thiên do hệ điều hành và tải máy. Mỗi lần nghiên cứu tạo một thư mục mới để tránh trộn số liệu cũ và mới.

Hướng dẫn chạy phần nghiên cứu bằng lệnh

Mở Anaconda Prompt, kích hoạt env ai, chuyển tới thư mục dự án. Không cần mở GUI MorphoLab để thực hiện các bước nghiên cứu. Cần Java JDK trong Fiji vì adapter được biên dịch trước khi chạy. Kiểm tra plugin có tại C:/Tools/Fiji/plugins/MorphoLibJ_-1.6.5.jar.

```
conda activate ai
cd /d D:\XU_LY_ANH
python -m morpholab dataset synthetic
python -m morpholab segment data\synthetic\synth_00.tif ^
  --truth data\synthetic\synth_00_gt.tif ^
  --out outputs\demo_shared --fiji
python -m morpholab segment data\synthetic\synth_00.tif ^
  --truth data\synthetic\synth_00_gt.tif ^
  --out outputs\demo_native --fiji-native
python -m morpholab benchmark data\synthetic\manifest.json ^
  --limit 1 --repeats 3 --out outputs\demo_benchmark
```

Lệnh segment xuất các TIFF giai đoạn, labels.tif, ảnh màu, bảng đối tượng và run.json. Khi có --truth, thêm metrics.json. Khi có --fiji, thư mục fiji chứa kết quả MorphoLibJ, chỉ số Java, dữ liệu đối chiếu và script Groovy. Chạy --fiji-native vào thư mục khác để giữ hai loại thí nghiệm riêng.

Kết quả mẫu mong đợi với cấu hình mặc định là bảy vật thể và F1 đối tượng bằng 1,000. IoU tiền cảnh không nhất thiết bằng 1 vì ngưỡng và làm trơn có thể làm biên khác nhãn hình học. Thời gian phụ thuộc máy, không dùng một con số thời gian cố định làm điều kiện pass.

Nếu lệnh báo không tìm thấy dữ liệu, chạy dataset synthetic hoặc dataset bbbc039. Nếu lỗi thiếu plugin, kiểm tra IJPB-plugins trong Fiji rồi cập nhật và mở lại. Nếu Java không biên dịch được, kiểm tra đường dẫn Fiji có thư mục java và javac; chi tiết xử lý lỗi và đường dẫn đầu ra có trong hướng dẫn chạy đi kèm.

Kịch bản demo và kiểm tra trực tiếp

Demo nên đi từ trường hợp hiểu được bằng mắt đến số liệu nhiều ảnh. Đầu tiên trình bày ảnh bảy đĩa, chỉ ra hai cặp chạm nhau. Chạy CCL để quan sát năm vùng, sau đó chạy hmax để quan sát bảy vùng. Mở mask, distance và marker để giải thích vì sao kết quả thay đổi. Không chỉ đưa một ảnh tô màu rồi khẳng định thuật toán tốt.

Tiếp theo mở script Groovy trong Fiji, chạy cùng elevation/marker/mask và lưu ảnh nhãn. So sánh Python với Fiji, giải thích rằng ID khác không đồng nghĩa khác vùng. Dùng metrics.json và fiji_metrics.csv để chỉ rõ hai nền tảng đều tính định lượng. Mở một ảnh BBBC039 để minh họa rằng ảnh thật có hình dạng khó hơn ảnh tổng hợp.

Sau đó trình bày bảng 50 ảnh test và hai biểu đồ validation. Nêu rõ CCL có thể tốt hơn Watershed trên một số điều kiện, native có pipeline khác, và F1 cần nhãn đúng. Chạy thử cấu hình min_area quá lớn hoặc marker quá nhiều để minh họa giới hạn, rồi khôi phục cấu hình mặc định.

Trong phần hỏi đáp, cần giải thích được: tại sao dùng −distance; tại sao cần marker; h ảnh hưởng gì; reconstruction khác opening thường ra sao; vì sao IoU tiền cảnh bằng nhau nhưng F1 khác; tại sao không so trực tiếp màu nhãn; và vì sao thời gian native không so trực tiếp với kernel Watershed.

Ứng dụng MorphoLab có thể dùng cho demo nếu thuận tiện. Các lệnh và script độc lập vẫn là đường tái lập chính của nghiên cứu. Một người trình bày không cần chạy lại toàn bộ 50 ảnh ngay tại lớp, nhưng phải chỉ được file cấu hình, CSV từng ảnh và lệnh đã dùng để tạo chúng.

Giới hạn và kết luận

Quy trình phù hợp hơn với ảnh có tiền cảnh nổi trên nền và vật thể có cấu trúc hình học rõ. Với ảnh tự nhiên nhiều màu, bóng đổ và nền phức tạp, một ngưỡng toàn cục có thể không tạo được mask đúng. Watershed không tự nhận biết tên loại đối tượng và không sửa được mọi lỗi từ mask.

Các kết quả validation chỉ gồm sáu ảnh đầu, chưa đại diện toàn bộ biến thiên. Các ảnh biến đổi nhiễu/mờ có liên hệ với ảnh gốc nên CI không thể xem như trên nhiều dữ liệu mới độc lập. Ground truth cũng có thể có sai số người gán nhãn. Nghiên cứu chưa đánh giá dữ liệu ngoài BBBC039 hay ảnh ba chiều.

Kết quả quan trọng là kiểm chứng được các toán tử tự cài theo quy ước rõ, có workflow Fiji thật và có đánh giá theo đối tượng. Hmax giảm chia thừa so với raw trên tập khảo sát, nhưng chưa vượt CCL ở mọi trường hợp. Fiji native đạt F1 cao hơn ở thí nghiệm hiện tại; cần phân tích khác biệt toàn pipeline trước khi gán nguyên nhân cho riêng Watershed.

Thử nghiệm tối ưu minh họa sự khác biệt giữa ý tưởng thuật toán và tốc độ triển khai: queue Python chưa nhanh hơn phép lặp dùng SciPy trên các mẫu được đo. Kết luận này hướng việc tối ưu tiếp vào các vòng lặp nóng và giao thức đo có kiểm soát. Báo cáo giữ các kết quả chưa thuận lợi để người đọc đánh giá đúng giới hạn.

Các sản phẩm chính gồm code Python, adapter và script Fiji, dữ liệu cấu hình, kết quả từng ảnh, bảng tổng hợp, báo cáo, slide và hướng dẫn chạy. Giao diện chỉ hỗ trợ thao tác và không thay thế việc hiểu, giải thích hoặc tái lập các bước thuật toán.

Tài liệu tham khảo

[1] Legland D, Arganda-Carreras I, Andrey P. MorphoLibJ integrated library and plugins for mathematical morphology with ImageJ. Bioinformatics, 2016. DOI: 10.1093/bioinformatics/btw413. Tham khảo thông tin thư viện và bài giới thiệu, không tuyên bố đã đọc toàn văn bài báo.

[2] MorphoLibJ manual v1.6.5 và mã nguồn chính thức. https://github.com/ijpb/MorphoLibJ ; https://imagej.net/plugins/morpholibj . Manual đã tải vào materials/downloads, phần reconstruction và watershed đã dùng để đối chiếu quy ước.

[3] Felzenszwalb PF, Huttenlocher DP. Distance Transforms of Sampled Functions. Theory of Computing 8, 2012. https://cs.brown.edu/people/pfelzens/dt/index.html . Tham chiếu thuật toán bao dưới parabol, cài Python riêng trong dự án.

[4] Broad Bioimage Benchmark Collection. BBBC039v1 Nuclei of U2OS cells in a chemical screen. https://bbbc.broadinstitute.org/BBBC039 . Dữ liệu CC0, ảnh và nhãn được tải theo nguồn chính thức.

[5] Caicedo JC. Ví dụ giải mã nhãn BBBC039. https://gist.github.com/jccaicedo/15e811722fca51e3ae90e8b43057f075 . Dự án giữ quy tắc thành phần cùng giá trị ở kênh đầu.

[6] Tài liệu API scikit-image về morphology và segmentation. https://scikit-image.org/docs/stable/api/skimage.morphology.html ; https://scikit-image.org/docs/stable/api/skimage.segmentation.html . Phiên bản thực chạy được khóa trong requirements-lock.txt.

[7] Slide học phần Xử lý ảnh PTIT về hình thái học, nguồn giảng viên cung cấp qua ảnh LMS. Bản tải cục bộ: materials/downloads/ptit_morphology.pdf. Danh sách liên kết và trạng thái đọc: materials/SOURCES.md.

[8] Mã nguồn và dữ liệu thí nghiệm của nhóm. https://github.com/jellalaz/XU_LY_ANH . Các CSV tại docs/evidence là nguồn số liệu của báo cáo; môi trường, tham số và thời gian chạy được lưu trong JSON tương ứng.

Đối chiếu yêu cầu và bàn giao

Yêu cầu	Minh chứng bàn giao
Lý thuyết đủ sâu	Định nghĩa, công thức, ví dụ và độ phức tạp trong báo cáo
Plugin Fiji	MorphoLibJ; shared và native; script Groovy
Cài đặt lại	morpholab/algorithms.py, pipeline.py; Java adapter
Định lượng hai nền tảng	Python metric và fiji_metrics.csv từ Java
Thực nghiệm và tối ưu	50 test × 4 cách; ablation, stress test; đo reconstruction
Báo cáo và presentation	Báo cáo này, bộ slide 10 trang, ghi chú thuyết trình
Hướng dẫn tái lập	submission/HUONG_DAN_CHAY_VA_TEST.md

Một đại diện nhóm nộp link theo hướng dẫn LMS, ghi rõ Nhóm 1 và tên thành viên. Deadline trong đề là 19/11; nhóm kiểm tra năm và giờ đóng cổng trực tiếp trên LMS trước khi nộp. Báo cáo và slide cần được từng thành viên đọc, chạy lại phần được phân công và giải thích được khi hỏi đáp.

Phân công dưới đây là phương án đề xuất, không phải xác nhận công việc cá nhân đã thực hiện: một người phụ trách lý thuyết và ví dụ tính; một người phụ trách Python và kiểm thử; một người phụ trách Fiji và script; một người phụ trách dữ liệu, metric và biểu đồ. Cả nhóm cùng rà soát phần phân tích và kết luận.

Trước khi nộp, mở thử báo cáo và slide trên máy trình chiếu, kiểm tra công thức tiếng Việt, liên kết repository và quyền đọc. Chạy lại ví dụ mẫu, một phép kiểm chứng Fiji và các unit test. Giữ kết quả đo đầy đủ trên repository hoặc thư mục chia sẻ; không cần đưa toàn bộ dataset tải về vào Git. Chưa có thao tác nộp LMS được thực hiện tự động trong dự án này.